PHOENIX — COMPREHENSIVE PRESENTATION AUDIT

Verified System Architecture, Data Flow, Real-Time Provenance & Defense Cheat Sheet

Executive Summary & Audit Scope

This audit reflects the **exact, verified source code and live runtime** of the PHOENIX system. Every UI element, backend calculation, database transaction, and computer vision pipeline stage has been traced line-by-line. Use this document as your authoritative guide during your presentation and evaluator Q&A.;

1. FRONTEND — COMPLETE UI COMPONENT AUDIT

Every visible element on the React Dashboard is audited below with its exact provenance classification and source file.

UI Element	Visible Label / Component	Classification	Source Component / File
PHOENIX Header	'PHOENIX' Badge & Logo	STATIC UI LABEL	DashboardPage.tsx (L200-213)
Building / Floor Info	'Phoenix Innovation Center · Ground Floor'	CONFIGURED / BACKEND REST API	DashboardPage.tsx (L215-219) via ApiClient.getBuilding()
Operating Layer Badge	'LAYER 1: NORMAL SURVEILLANCE' / 'LAYER 2: EMERGENCY'	CALCULATED FROM REAL-TIME DATA	DashboardPage.tsx (L221-244) driven by active Incident state
API Health Status	'API: HEALTHY' (Green / Yellow dot)	REAL-TIME (REST Polled)	DashboardPage.tsx (L249-259) via /api/v1/system/health
WebSocket Status	'WS STREAM: LIVE' / 'DISCONNECTED'	REAL-TIME	WebSocketContext.tsx & DashboardPage.tsx (L261-271)
RBAC User Menu	e.g., 'usr-admin-01 (SUPERADMIN)' + Sign Out	REAL-TIME (JWT Auth)	AuthContext.tsx & DashboardPage.tsx (L273-292)
Operational Event Banner	'System Operational Event: HAZARD CONFIRMED...'	REAL-TIME (WebSocket Event)	DashboardPage.tsx (L296-325) from INCIDENT_STATUS_CHANGED
Severity / Incident Banner	'■ HIGH SEVERITY: Detected FIRE in South Exit Corridor'	CALCULATED FROM REAL-TIME DATA	DashboardPage.tsx (L333-370) from active incident
Primary Hazard CCTV Feed	Auto-focused CCTV video with red hazard border & badge	REAL-TIME / CONFIGURED STREAM	CameraFeed.tsx & DashboardPage.tsx (L387-407)
Monitoring Feeds Drawer	Collapsible feeds (North Corridor, Main Hall West, East Exit)	CONFIGURED CAMERAS / LIVE VIDEO	DashboardPage.tsx (L410-451)
YOLO Perception Telemetry	Active detections: Class, Confidence %, Bounding Box	REAL-TIME (Live YOLOv11)	DashboardPage.tsx (L478-501) from DETECTION_OBSERVED event
Smoke Growth Telemetry	Smoke Area (px²), Growth Rate (px²/s), Trend ('GROWING')	CALCULATED FROM REAL-TIME DATA	DashboardPage.tsx (L503-530) from SMOKE_GROWTH_ALERT
Spatial Risk Assessment	Global Risk Level ('HIGH') & per-node risk percentages	CALCULATED FROM REAL-TIME DATA	DashboardPage.tsx (L532-554) from RISK_UPDATED event
Spatial Map & Blueprint	SVG Layout: Rooms, Corridors, Exits, Dynamic Heatmap halos	CONFIGURED TOPOLOGY + REAL-TIME OVERLAY	EvacuationMap.tsx (L150-363)
Evacuation Origin Selector	Dropdown of rooms/nodes (e.g., 'Room 101 (room_101)')	CONFIGURED / INTERACTIVE	DashboardPage.tsx (L562-584)
Designated Safe Exit & ETA	'exit_east · 30.7s' (or exit_south depending on hazard)	CALCULATED FROM REAL-TIME DATA	DashboardPage.tsx (L586-602) from Dijkstra & ETA engine

Computed Path Sequence	'room_101 → node_c1 → node_n1 → exit_east'	CALCULATED FROM REAL-TIME DATA	DashboardPage.tsx (L604-620)
Route Reasoning Breakdown	Decomposed bars: Distance, Hazard, Occupancy, Blockage	CALCULATED FROM REAL-TIME DATA	DashboardPage.tsx (L629-677) from RouteRecommendation

2. BACKEND — COMPLETE RUNTIME DATA FLOW PIPELINE

The live pipeline executes continuously in a background asyncio task managed by app.main:lifespan:

- **1. VIDEO SOURCE:** Simulated CCTV mp4 stream (e.g., videos/test2.mp4) loaded via OpenCV VideoCapture in VideoFrameSource (5 FPS loop).
- **2. FRAME INGESTION:** Continuously decodes BGR numpy frames at ~2 FPS in ContinuousPerceptionPipeline._perception_loop() in continuous_pipeline.py.
- **3. YOLO DUAL INFERENCE:** YOLOAdapter.detect() executes fire_smoke_best.pt (fire/smoke) and yolo11n.pt (persons). Extracts confidence and bounding boxes.
- **4. SPATIAL HOMOGRAPHY:** PlanarHomographyMapper.image_to_floorplan() projects bbox ground point (x_center, y_max) into 2D metric floor coordinates (meters).
- **5. PERSISTENCE & WS BROADCAST:** Persists batch detections to SQLite detections table; publishes DETECTION_OBSERVED domain event to WebSocket EventBus.
- **6. TEMPORAL SMOKE ANALYSIS:** TemporalSmokeAnalyzer.process_observations() maintains sliding time window (10s) to calculate **Δarea / Δt (px²/s)** and classifies trend (RAPID_GROWTH, MODERATE_GROWTH, STABLE). Emits SMOKE_GROWTH_ALERT.
- **7. INCIDENT MANAGEMENT:** When fire/smoke is detected, automated incident is created/updated in SQLite (IncidentRepository) and immutable audit log (AuditEventRepository). Emits INCIDENT_STATUS_CHANGED.
- **8. DETERMINISTIC RISK ENGINE:** DeterministicRiskEngine.calculate_risk() computes per-node risk using exponential spatial distance decay: *Risk = BaseWeight × Conf × GrowthMultiplier × exp(-0.5 × Dist)*. Emits RISK_UPDATED.
- **9. OCCUPANCY ESTIMATION:** ApproximateOccupancyEstimator.estimate_node_occupancies() aggregates person detections mapped to nearest navigation nodes.
- **10. PREDICTIVE BLOCKAGE:** PredictiveBlockageEngine.predict_blockage_scores() forecasts corridor blockage: *0.3 × CurrentHazard + 0.7 × SmokeGrowthSignal × CurrentHazard*.
- **11. MULTI-FACTOR ROUTING:** MultiFactorRoutingEngine.compute_route() executes Dijkstra over NetworkX graph minimizing: *0.35 × Dist + 0.40 × Hazard + 0.10 × Occupancy + 0.15 × PredictedBlockage*.
- **12. ETA ESTIMATION:** DeterministicETAEstimator.calculate_eta() calculates travel time in seconds: *Length / (1.2 m/s × StairsFactor × CongestionFactor × SmokeFactor)*.
- **13. SQLITE PERSISTENCE:** Saves RouteRecommendation to SQLite table; dispatches ROUTE_CHANGED event over WebSocket.
- **14. REACT UI RENDERING:** React WebSocket hook updates state, dynamically flips UI from Layer 1 to Layer 2, draws heat halos, updates cost breakdown and route path in real time.

3. REAL-TIME BEHAVIOR VS 4. SQLITE PERSISTENCE

Exact mapping of live runtime execution vs SQLite database persistence in phoenix.db:

Entity / State	Storage Classification	SQLite Table Name	Repository & Runtime Operations
Cameras	PERSISTED & CONFIGURED	cameras	SQLiteCameraRepository.save() on startup, list_by_building()
Buildings & Floors	PERSISTED & CONFIGURED	buildings, floors	SQLiteBuildingRepository, SQLiteFloorRepository

YOLO Detections	PERSISTED (Real-Time)	detections	SQLiteDetectionRepository.save_batch() per live frame
Incidents	PERSISTED (Real-Time)	incidents	SQLiteIncidentRepository.save() on declaration/resolution
Route Recommendations	PERSISTED (Real-Time)	route_recommendations	SQLiteRouteRecommendationRepository.save() on re-routing
Audit Trail Events	PERSISTED (Immutable)	audit_events	SQLiteAuditEventRepository.record_event() with idempotency key
Floorplan Models	PERSISTED & CONFIGURED	floorplans	SQLiteFloorplanRepository.save()
Risk Snapshots	MEMORY ONLY / WS EVENT	*Not persisted to table*	Streamed live via RISK_UPDATED WebSocket domain event
Temporal Smoke Observations	MEMORY ONLY / WS EVENT	*In-memory sliding deque*	Streamed live via SMOKE_GROWTH_ALERT WebSocket event

5. WHAT IS CONFIGURED VS REAL-TIME & 6. SPATIAL TOPOLOGY

Definition Clarification: Distinguish clearly between *Configured Test Environment* (legitimate engineering parameters) and *Real-Time Computed State*.

- **50m × 40m Floor Dimensions: CONFIGURED TEST ENVIRONMENT.** Defined in app/spatial/floorplan_parser.py and continuous_pipeline.py (L150-151) as the physical architectural boundary of Ground Floor Main Lobby.
- **Navigation Nodes (room_101, node_c1, node_n1, node_s1, exit_south, exit_east): CONFIGURED TOPOLOGY.** Defined in FloorplanProcessor.create_synthetic_architectural_floorplan(). In production, these can be extracted automatically by CV floorplan analysis in process_floorplan_image().
- **Camera Mappings (CAM-SOUTH-01, CAM-NORTH-01, CAM-HALL-01): CONFIGURED SURVEILLANCE MATRIX.** Defined in continuous_pipeline.py (L157-197), binding camera channels to monitored node regions and CCTV video paths.
- **Homography Calibration Points: CONFIGURED 4-POINT CALIBRATION.** Defined in continuous_pipeline.py (L216-224), mapping (0,0)-(3840,2160) video frame pixels to (0,0)-(50,40) metric floor meters. *If calibration is missing, the system gracefully falls back to pixel-relative mode and flags PROVENANCE: UNCALIBRATED.*
- **Incident Creation, Risk, Occupancy & Route: GENUINE REAL-TIME COMPUTATION.** Detections emerge from real YOLO inference on video frames; risk decays across metric distance; Dijkstra dynamically reroutes when South corridor becomes hazardous.

7. THE 4 KEY FEATURES — DEEP TECHNICAL AUDIT

A. Smoke Velocity / Growth Analysis

- **Input:** Bounding box areas of DetectionType.SMOKE across successive video frames.
- **Calculation:** Sliding window rate of change: *GrowthRate = (Area_latest - Area_earliest) / dt*. Exponential Moving Average smoothed with α=0.30.
- **Classification:** >250 px²/s = RAPID_GROWTH, >50 px²/s = MODERATE_GROWTH, ≤50 px²/s = STABLE.
- **Frontend:** Displayed in dedicated telemetry card with live area (px²), growth rate (px²/s), and status badge.
- **Limitation:** Measures 2D pixel area expansion in camera view plane, not 3D physical volumetric mass velocity.

B. Occupancy Penalty

- **Input:** DetectionType.PERSON visual detections with ground-projected floor coordinates.
- **Calculation:** Euclidean distance mapping of person points to nearest node. Normalized in Dijkstra cost: *Cost_occ = 0.10 × min(1.0, count / 20)*.

- **ETA Effect:** Congestion slowdown scaling up to 50% when node headcount exceeds 5 persons.
- **Limitation:** Visual headcount estimation based on camera line-of-sight; does not simulate high-density crowd crush physics.

C. Dynamic Risk Heatmap

- **Input:** Fire/smoke detections, mapped metric coordinates, base weights (Fire=1.0, Smoke=0.6), temporal growth multiplier.
- **Calculation:** Deterministic spatial decay: *Risk(node) = BaseWeight × Confidence × GrowthMult × exp(-0.50 × Distance_meters)*.
- **Frontend:** Rendered as pulsing multi-ring heat halos around nodes on SVG Blueprint (Red ≥70%, Orange ≥30%, Yellow >5%).
- **Limitation:** Employs 2D Euclidean exponential decay rather than fluid-dynamic CFD smoke propagation.

D. ETA Counter

- **Input:** Dijkstra path edges, edge lengths in meters, active occupancy, and corridor hazard scores.
- **Calculation:** *Time = Σ [Length / (1.2 m/s × StairsFactor × CongestionFactor × SmokeVisibilityFactor)]*.
- **Real-Time Response:** Continuously updates (e.g., 30.7s) when origin changes or hazard causes path rerouting.
- **Limitation:** Assumes average nominal walking speed of 1.2 m/s; individual mobility differences are not modeled.

8. STEP-BY-STEP DEMONSTRATION STORYBOARD

Follow this structured narrative during your live demonstration:

1. Initial State (Normal Surveillance): Open Dashboard. Point out *LAYER 1: NORMAL SURVEILLANCE* badge, 4 active CCTV streams, live API health (Healthy), and WebSocket stream (Live).

2. Video Ingestion & Perception: Explain that OpenCV decodes frames at 5 FPS while dual YOLO models continuously inspect frames. No hazard is present, so incident is NULL.

3. Fire / Smoke Ignition: As simulated video progresses into fire/smoke frames on CAM-SOUTH-01, YOLO detects fire (conf >85%).

4. Emergency Mode Transition: System automatically creates incident INC-... in SQLite and emits INCIDENT_STATUS_CHANGED. The UI seamlessly transitions to *LAYER 2: EMERGENCY DECISION-SUPPORT*.

5. Live Telemetry Activation: Primary hazard CCTV feed is isolated on the left; right column renders real-time YOLO bounding boxes, Smoke Growth Rate (+82 px²/s, GROWING), and Node Risk percentages.

6. Dynamic Rerouting & ETA: Explain that South corridor (node_s1 → exit_south) is blocked due to extreme hazard risk. Dijkstra algorithm automatically computes safe egress through North corridor to **Exit East (30.7s ETA)**.

7. Route Reasoning Breakdown: Show the decomposed cost bars: Distance, Hazard Exposure, Occupancy Density, and Predicted Blockage.

8. Incident Resolution: When hazard video clears (30 clean cycles), backend automatically sets incident to RESOLVED, writes audit log, and restores dashboard to Layer 1.

9. VERIFIED TECHNOLOGY STACK

Category	Technology / Library	Usage in PHOENIX
Frontend	React 18, TypeScript, Vite, SVG Blueprint	Single-page command dashboard with zero external heavy UI libraries
Backend API	FastAPI, Python 3.10+, Uvicorn, asyncio	REST API contracts, async lifespan worker, background perception loop
Real-Time Comms	Native WebSockets, In-Memory EventBus	Low-latency event streaming (detections, smoke alerts, risk, route)
Computer Vision	Ultralytics YOLOv11, OpenCV (cv2)	Dual-model inference (fire_smoke_best.pt, yolo11n.pt), frame decoding
Spatial Mapping	Planar Homography, NumPy, OpenCV	Perspective transformation mapping camera pixels to floorplan meters
Graph & Routing	NetworkX, Dijkstra Algorithm	Multi-factor cost-weighted evacuation path optimization
Database	SQLite (WAL mode), Foreign Keys	ACID persistence for incidents, routes, detections, cameras, audit log
Security / Auth	JWT (python-jose), Passlib (bcrypt)	Role-Based Access Control (Superadmin, Responder, Manager, Occupant)

10. PRESENTATION CHEAT SHEET & EVALUATOR DEFENSE

WHAT YOU CAN CONFIDENTLY SAY:

1. 'PHOENIX implements continuous, dual-model YOLOv11 computer vision detecting fire, smoke, and occupants simultaneously on live video streams.'
2. 'Spatial mapping uses planar homography projection to convert camera image pixels directly into real-world metric coordinates.'
3. 'Evacuation routing is deterministic, executing Dijkstra shortest path over a dynamic NetworkX graph weighted by distance, hazard, occupancy, and predicted blockage.'
4. 'All incident declarations, routing changes, detections, and resolution events are persisted with an immutable audit trail in SQLite.'
5. 'The frontend is a reactive React dashboard connected via WebSockets, dynamically transitioning between Normal Surveillance and Emergency Decision Support.'

WHAT YOU SHOULD NOT CLAIM (HONEST BOUNDARIES):

1. Do NOT claim the smoke growth analyzer calculates 3D physical fluid mass velocity — it measures 2D pixel area expansion rate in camera space.
2. Do NOT claim occupancy modeling solves complex high-density panic stampede crowd crush — it is an approximate visual headcount penalty.
3. Do NOT claim the floorplan is magically guessed without inputs — floor dimensions (50m×40m) and camera calibrations are configured architectural ground-truths.
4. Do NOT claim PHOENIX is a certified NFPA-compliant life safety system — it is an intelligent emergency decision-support research prototype.

ONE-SENTENCE ARCHITECTURE:

'PHOENIX is an intelligent emergency decision-support platform that fuses real-time dual YOLO perception, temporal smoke growth analysis, planar homography spatial mapping, and multi-factor Dijkstra routing to generate dynamic, life-saving evacuation guidance.'

60-SECOND BACKEND DEFENSE SPEECH:

'On the backend, PHOENIX runs an asynchronous continuous perception loop. Frames are ingested from CCTV streams and passed to dual YOLOv11 models for fire, smoke, and person detection. Ground points are projected onto metric floor coordinates via homography calibration matrices. A temporal analyzer tracks smoke area growth rates to forecast corridor blockage. The deterministic risk engine applies exponential distance decay to update node risks, and a multi-factor Dijkstra routing engine calculates optimal evacuation paths with accurate ETAs. State is persisted in SQLite with atomic audit trails, and events are broadcasted over authenticated WebSockets.'

60-SECOND FRONTEND DEFENSE SPEECH:

'The frontend is built in React and TypeScript. In Layer 1 Normal Surveillance mode, it displays the CCTV camera matrix and standby spatial map. When the backend detects a genuine hazard, the dashboard automatically transitions to Layer 2 Emergency Decision Support. The hazard camera is isolated, live YOLO bounding boxes and smoke growth rates stream to telemetry cards, the SVG blueprint renders real-time risk heat halos, and the safest exit route is highlighted with step-by-step corridor sequences, ETA counters, and decomposed cost reasoning bars.'